Documentación técnica - Omnara E-Commerce

Análisis del estado actual del repositorio `jp-software-dev/omnara-ecommerce`, rama `main`. Generado a partir del código fuente y del esquema en vivo de Supabase, no de suposiciones.

1. Resumen del proyecto

Omnara es un **marketplace multi-vendedor** (proyecto escolar IUEM) construido con Next.js 16 (App Router) y Supabase. Varios negocios independientes venden bajo un mismo catálogo, carrito y checkout. Modela un catálogo real - ropa, calzado, electrónica, accesorios y hogar -, cada categoría con sus propios atributos específicos, detrás de un panel admin/vendedor basado en roles y un checkout de Stripe arquitectónicamente bloqueado para nunca procesar un cargo real.

Modelo marketplace: la tabla `businesses` (un negocio por dueño) representa a cada vendedor; `products.vendor_id` y `order_items.vendor_id` ligan productos y ventas al vendedor. Un carrito multi-negocio genera **un solo pedido** y el dinero no se reparte (no hay Stripe Connect).

2. Stack tecnológico

Capa	Tecnología	Versión
Framework	Next.js (App Router)	16.3.0
UI	React	19.2.8
Lenguaje	TypeScript	^5
Estilos	Tailwind CSS	v4 (`@tailwindcss/postcss`)
Componentes	shadcn/ui sobre Base UI (`@base-ui/react`)	^1.7.0
Backend / BD	Supabase (Postgres, Auth, Storage, RLS)	`@supabase/ssr` ^0.12.4, `@supabase/supabase-js` ^2.112.1
Pagos	Stripe (modo prueba)	^22.4.0
Estado cliente	Zustand	^5.0.14
Datos/cache cliente	TanStack Query	^5.101.4
i18n	next-intl	^4.13.5
Validación	Zod	^4.4.3
Notificaciones UI	sonner (toasts)	^2.0.7
Temas	next-themes	^0.4.6
Hosting	Vercel (build con Turbopack) + Supabase	-

Nota Next.js 16: este proyecto usa convenciones nuevas de Next 16 que rompen con versiones anteriores - el middleware ya no es `middleware.ts` con `export function middleware`, sino `src/proxy.ts` con `export async function proxy(request)`. `AGENTS.md` en la raíz lo advierte explícitamente ("This is NOT the Next.js you know") y remite a `node_modules/next/dist/docs/` antes de escribir código nuevo, porque las APIs pueden diferir de lo que un asistente conoce por entrenamiento.

3. Arquitectura general

- **App Router** con un segmento dinámico `[locale]` en la raíz de casi todas las rutas (`/es/...`, `/en/...`), vía `next-intl` con `localePrefix: "always"` (`src/i18n/routing.ts`).
- **Route groups** para separar layouts sin afectar la URL:
- `(shop)` - tienda pública (home, categoría, producto, carrito, checkout, buscar, ayuda, contacto, términos, privacidad).
- `(account)` - área de cliente autenticado (`/cuenta`, `/cuenta/pedidos`).
- `(admin)` - panel admin/vendedor (`/admin/*`).

- `login`, `signup`, `recuperar`, `restablecer` viven directamente bajo `[locale]/` (fuera de los grupos anteriores), con su propio layout minimalista.
- **Server Components por defecto**; los componentes interactivos (formularios, selects optimistas, carrito) están marcados `"use client"`.
- **Server Actions** (`"use server"`) para todas las mutaciones - no hay una capa de API REST propia aparte del webhook de Stripe.
- **Un único endpoint de API route real:** `src/app/api/webhooks/stripe/route.ts` (recibe eventos de Stripe).

4. Estructura de carpetas relevante

```
src/
  app/
    [locale]/
      (shop)/          -> tienda pública
      (account)/        -> cuenta del cliente
      (admin)/           -> panel admin/vendedor
      login, signup, recuperar, restablecer
    api/webhooks/stripe/  -> webhook de Stripe
  components/
    ui/                -> primitivos shadcn/ui (button, select, table, accordion, dialog, card, field, badge...)
    shop/                -> componentes de tienda (header, footer, carrito, checkout, reseñas, etc.)
    auth/                -> formularios de login/signup/recuperación
    admin/               -> componentes exclusivos del panel admin (p. ej. gráfica de barras)
  lib/
    supabase/            -> clients (browser/server/service-role), queries, admin-queries, tipos generados, auth-helpers
    stripe.ts, shipping.ts, format.ts, category-attributes.ts, size-charts.ts, color-swatches.ts, utils.ts
  stores/                -> Zustand: cart-store.ts, currency-store.ts
  i18n/                  -> routing.ts, navigation.ts, request.ts
  messages/              -> es.json, en.json (namespace `common`/`home`; el resto de textos están hardcodeados inline po
  proxy.ts               -> middleware de Next 16 (sesión de Supabase + i18n)
```

5. Internacionalización

- `next-intl` con dos locales: `es` (default) y `en`, prefijo de URL siempre presente (`/es/...`, `/en/...`).
- El archivo `messages/*.json` solo cubre el namespace `common` (marca, buscar, carrito, cuenta, favoritos) y `home`. **El resto de la interfaz no usa el sistema de mensajes de next-intl** - cada página/componente define un objeto `t` local con el patrón `locale === "en" ? "..." : "..."`. Es una decisión de diseño consistente en todo el proyecto, no un descuido puntual.
- Las pantallas de auth (`login-form.tsx`, `signup-form.tsx`, `forgot-password-form.tsx`, `reset-password-form.tsx`) leen el idioma con `useLocale()` y usan el mismo objeto `t` en línea que el resto de la interfaz.
- `pickLocale(jsonValue, locale)` (`src/lib/format.ts`) extrae el string correcto de un campo JSONB bilingüe (`{es, en}`) usado en `products.name`, `categories.name`, etc., con fallback a `es`.

6. Modelo de datos (Supabase / Postgres)

Proyecto Supabase: `omnara-ecommerce` (`jqgfsbtevkofhwlaxutt`, región `us-west-2`, Postgres 17).

Tablas (`public`)

Tabla	Propósito	Columnas clave
`profiles`	Perfil 1:1 con `auth.users`	`id` (FK a `auth.users`), `full_name`, `phone`, `role` (`admin`/`vendor`/`customer`, default `customer`), `preferred_locale`, `preferred_currency`
`addresses`	Libreta de direcciones del comprador (se usa en `/cuenta/direcciones` y en el checkout)	`user_id`, `label`, `recipient_name`, `phone`, `line1/2`, `city`, `state`, `postal_code`, `country` (MX/US), `type`, `is_default` (una sola por usuario y tipo; máximo 10 por usuario)
`categories`	Árbol de categorías (`parent_id` autorreferencial)	`slug`, `name`/`description` (jsonb bilingüe), `position`

Tabla	Propósito	Columnas clave
warehouses	Almacenes (1 seed activo)	name, address, is_default
businesses	Negocio de cada vendedor	owner_id (FK a profiles, **unique**: un negocio por dueño), slug (unique), name, description, logo_url, banner_url, is_active, created_at, updated_at
products	Catálogo	vendor_id, category_id, slug, name/description_short/description_long (jsonb bilingüe), base_price_mxn_cents, status (draft/active/archived), is_featured, attributes (jsonb, **específico por categoría**, ver §6.1)
product_variants	SKU real (talla/color/precio/stock)	product_id, sku (unique), size, color, price_override_mxn_cents, stock_quantity, low_stock_threshold (default 5)
product_images	Imágenes por producto o por variante	product_id, variant_id (nullable), url, alt_text, position
inventory_movements	Auditoría de movimientos de stock	variant_id, warehouse_id, quantity_change, reason (restock/sale/adjustment/return), created_by
carts / cart_items	Carrito persistido en BD (existe, pero la app usa Zustand+localStorage como fuente real del carrito - ver §11)	carts.user_id/session_id/currency; cart_items.cart_id, variant_id, quantity
orders	Pedido	order_number (unique), user_id (nullable -> invitado), status (pending/paid/fulfilled/cancelled/refunded), currency, subtotal_cents, shipping_cents, discount_cents, promo_code, total_cents, shipping_address_snapshot (jsonb), tracking_number, payment_method, stripe_payment_intent_id (unique de facto por lookup); customer_email (correo del comprador, incluidos invitados)
order_items	Líneas de pedido, con snapshot histórico	order_id, variant_id, vendor_id, product_name_snapshot/variant_attrs_snapshot (jsonb), unit_price_cents, quantity; shipped_at, tracking_number (envío **por vendedor**)
reviews	Reseñas de producto	product_id, user_id, rating (1-5), title, comment, is_verified_purchase (**calculado por un trigger**), is_hidden (moderación)
promo_codes	Códigos de descuento	code (unique), discount_type (percentage/fixed), discount_value, min_order_cents, expires_at, usage_limit, times_used, is_active
wishlists	Favoritos	user_id, product_id
app_settings	Config global, fila única (id = 1)	usd_exchange_rate (default 18.0), free_shipping_threshold_mxn_cents (default 150000 = $1,500 MXN)
contact_messages	Buzón de contacto	name, email, subject, message, status (new/read)
seller_applications	Solicitudes para vender	user_id, business_name, description, phone, document_path (bucket privado seller-documents), status (pending/approved/rejected), rejection_reason, reviewed_by

Tabla	Propósito	Columnas clave
`return_requests`	Devoluciones (una por artículo)	`order_item_id` (unique), `order_id`, `user_id`, `vendor_id`, `reason`, `details`, `status` (`requested`/`approved`/`rejected`/`escalated`/`closed`/`refunded`), `refund_amount_cents`, `currency`, `stripe_refund_id`
`return_messages`	Hilo de cada devolución	`return_id`, `author_id`, `author_role` (`buyer`/`vendor`/`admin`), `body`
`product_questions`	Preguntas y respuestas públicas	`product_id`, `user_id`, `question`, `answer`, `answered_by`, `answered_at`, `is_hidden`
`email_outbox`	Cola de notificaciones por correo	`to_email`, `locale`, `template`, `payload`, `status` (`pending`/`sent`/`failed`), `attempts`, `last_error`
`admin_audit_log`	Bitácora de cambios de administradores (solo se agrega)	`actor_id`, `table_name`, `record_id`, `action`, `changes` (solo columnas que cambiaron)
`data_requests`	Solicitudes de eliminación de cuenta	`user_id`, `kind`, `status` (`pending`/`done`/`rejected`), `admin_note`, `resolved_by`
`business_ratings` (vista)	Calificación de cada negocio: promedio y número de reseñas visibles de sus productos (`security_invoker`)	`business_id`, `review_count`, `avg_rating`

Convención de dinero: todos los montos se guardan como enteros en **centavos de MXN** (`*_cents`), nunca floats. `promo_codes.discount_value` es la única excepción con doble semántica: 0-100 para `percentage`, centavos MXN para `fixed` (convención documentada en el código, no forzada por un `check`).

Convención bilingüe: todo texto orientado al usuario que vive en BD (`name`, `description*`) se guarda como JSONB `{ "es": "...", "en": "..." }`, nunca como filas separadas por idioma.

6.1. Atributos por categoría (`src/lib/category-attributes.ts`)

`products.attributes` es JSONB de forma libre; el frontend define qué campos mostrar/editar según la categoría:

Categoría	Campos
`ropa`	material, género (Hombre/Mujer/Unisex), temporada
`calzado`	material, género, tipo de suela
`electronica`	marca, modelo, año, garantía (meses)
`accesorios`	marca, material
`hogar`	marca, material, dimensiones

Esto alimenta tanto el formulario admin de producto como la sección "Características" de la ficha de producto - un mismo origen de verdad, sin esquema rígido por tabla.

6.2. Funciones de Postgres (`public`/`private`)

Todas `SECURITY DEFINER`:

Función	Uso
`private.is_admin()`	Helper de RLS - `true` si `auth.uid()` tiene `role = 'admin'`
`private.is_vendor()`	Helper de RLS
`private.is_admin_or_vendor()`	Helper de RLS (usado por `warehouses_read`)
`private.is_order_item_vendor(order_id)`	Helper de RLS - ¿el usuario actual vendió algún ítem de ese pedido?
`private.order_owner_matches(order_id)`	Helper de RLS
`handle_new_user()`	Trigger que crea la fila en `profiles` al registrarse en `auth.users`

Función	Uso
`admin_list_users()`	Devuelve `id, email, full_name, role, created_at` uniendo `profiles` + `auth.users` (que no es accesible vía PostgREST); solo retorna filas si el llamante es admin. Permite listar usuarios con email **sin necesitar** SUPABASE_SERVICE_ROLE_KEY en runtime. EXECUTE revocado a `anon`, otorgado a `authenticated`.

6.2b. Funciones añadidas (todas `SECURITY DEFINER`, con `search_path` fijo)

Las escrituras sensibles no tienen políticas de INSERT/UPDATE: pasan por funciones que verifican quién llama.

Función	Uso
`vendor_ship_items(order, tracking)`	El vendedor marca sus artículos como enviados; el pedido pasa a enviado cuando no queda nada pendiente
`approve_seller_application(id, slug)` / `reject_seller_application(id, reason)`	Aprobar crea el negocio, promueve al usuario y cierra la solicitud en una transacción
`request_return`, `decide_return`, `escalate_return`, `admin_resolve_return`, `mark_return_refunded`, `add_return_message`	Flujo de devoluciones; el monto del reembolso se calcula en la base
`ask_question`, `answer_question`, `moderate_question`	Preguntas y respuestas; límite de 5 preguntas por hora
`moderate_review(id, hidden)`	Ocultar o restaurar una reseña (solo admin)
`submit_contact_message(...)`	Única vía para escribir en `contact_messages`; valida y limita la frecuencia
`request_account_deletion()` / `resolve_data_request(...)`	Solicitudes de eliminación de cuenta
`private.user_purchased`, `private.has_purchased`	Comprueban compras reales (política de reseñas y trigger de la marca verificada)
Disparadores `email_*`	Encolan correos; cada uno atrapa sus errores para no bloquear la operación que los origina
Disparadores `audit_*`	Escriben en la bitácora; solo registran acciones de administradores

6.3. Row Level Security (RLS) - política completa

RLS está **habilitado en todas las tablas** de `public`. No hay ninguna tabla de negocio sin RLS.

Tabla	Regla efectiva
`profiles`	SELECT/UPDATE: propio registro o admin
`addresses`	ALL: propio registro o admin
`categories`	SELECT: público; INSERT/UPDATE/DELETE: admin
`warehouses`	SELECT: admin o vendor; INSERT/UPDATE/DELETE: admin
`businesses`	SELECT: público; INSERT/DELETE: admin; UPDATE: dueño (`owner_id = auth.uid()`) o admin
`products`	SELECT: `status = active` o dueño (`vendor_id`) o admin; INSERT/UPDATE/DELETE: dueño o admin
`product_variants` / `product_images`	Mismo patrón que `products`, verificado vía join al producto padre
`inventory_movements`	SELECT/INSERT: dueño del producto (vendor) o admin
`carts` / `cart_items`	ALL: dueño del carrito
`orders`	SELECT: comprador, admin, o vendor con algún ítem en el pedido; UPDATE: **solo admin** (el vendedor solo toca sus artículos mediante `vendor_ship_items`)
`order_items`	SELECT: vendor del ítem, admin, o dueño del pedido
`reviews`	SELECT: público salvo reseñas ocultas (las ven su autor y los admins); INSERT: solo compradores del producto (`reviews_insert_buyers`); UPDATE/DELETE: propio o admin; un trigger fija la marca verificada y solo un admin cambia `is_hidden`

Tabla	Regla efectiva
`promo_codes`	ALL: **solo admin** - el storefront no puede leer esta tabla con el cliente normal (ver §9.2)
`wishlists`	ALL: dueño
`app_settings`	SELECT: público; UPDATE: admin
`contact_messages`	Sin INSERT directo (se escribe con `submit_contact_message`); ALL: solo admin
`seller_applications`	SELECT: propio o admin; INSERT: solo clientes, para sí mismos y como pendiente; sin UPDATE/DELETE
`return_requests` / `return_messages`	SELECT: comprador, vendedor del artículo o admin; sin escritura directa
`product_questions`	SELECT: público salvo ocultas (las ven su autor y los admins); sin escritura directa
`email_outbox`, `admin_audit_log`	SELECT: solo admin; sin escritura directa (las escriben disparadores o el rol de servicio)
`data_requests`	SELECT: propio o admin; sin escritura directa
`businesses` (clave foránea)	`products.vendor_id` referencia `businesses.owner_id`: un vendedor sin negocio no puede crear productos y un negocio con productos no se puede borrar

Principio de diseño explícito en el código: "RLS ya scopea esto - un vendor consultando la base de datos directamente obtiene el mismo alcance que la UI muestra, porque la base de datos es la que lo aplica." Ninguna Server Action filtra manualmente por rol antes de tocar la BD; todas usan el cliente autenticado normal (`createClient()`) y dejan que RLS decida qué filas son visibles o mutables.

7. Autenticación y sesión

- Supabase Auth (email + contraseña). Flujo de registro con confirmación por correo (`signUp` -> mensaje "revisa tu correo").
- Recuperación de contraseña: `resetPasswordForEmail` -> `/recuperar`, enlace de correo lleva a `/restablecer` -> `updateUser({ password })`. Flujo cliente-only, sin ruta de callback en servidor (implica que el proyecto de Supabase usa el flujo implícito de detección de sesión en URL, no PKCE con code exchange).
- Sesión mantenida vía cookies HTTP-only, refrescada en cada request por `updateSession()` (`src/lib/supabase/middleware.ts`), invocada desde `src/proxy.ts`.
- **Login con Google:** botón presente en la UI pero es un stub (`toast` "se conecta en la siguiente fase") - requiere credenciales OAuth de Google Cloud Console que no pueden generarse desde el código; fuera de alcance por diseño.
- `requireRole(allowedRoles, locale)` (`src/lib/supabase/auth-helpers.ts`) - guard de servidor usado en todas las páginas de `(admin)`: redirige a `/login` si no hay sesión, a `/` si el rol no matchea.

8. Pagos (Stripe) - flujo completo

8.1. Cliente Stripe (`src/lib/stripe.ts`)

`getStripe()` construye el cliente **rechazando cualquier clave que no empiece con** `sk_test_` **o** `rk_test_` - es arquitectónicamente imposible procesar un cargo real con este código, sin importar qué clave se configure.

8.2. Creación de sesión de checkout (`(shop)/checkout/actions.ts`)

1. Recalcula el precio de cada línea **desde la base de datos** (nunca confía en el precio que manda el cliente).
2. Convierte a USD si aplica, usando `app_settings.usd_exchange_rate` como única fuente de verdad (misma tasa para lo que se muestra y lo que se cobra).
3. Calcula envío: gratis si el subtotal >= `app_settings.free_shipping_threshold_mxn_cents`; si no, tarifa plana `FLAT_SHIPPING_MXN_CENTS = 9900` ($99 MXN), convertida a la moneda de la sesión.

4. Si viene un `promoCode`, lo revalida server-side contra `promo_codes` (vía `validatePromoCode`, con `createServiceRoleClient()` - ver §9.2) y, si es válido, crea un **cupón Stripe efímero de un solo uso** (`stripe.coupons.create({ amount_off, duration: "once" })`) en vez de restar el descuento manualmente a cada línea.
5. Construye los `line_items` (uno por variante + uno de "Envío" si aplica) y crea la `checkout.session` con `shipping_address_collection` activo y metadata: `user_id`, `locale`, `shipping_cents`, `discount_cents`, `promo_code`, `cart_items` (JSON serializado de variant/qty/precio).
6. Redirige al `session.url` de Stripe.

8.3. Webhook (`src/app/api/webhooks/stripe/route.ts`)

- Verifica la firma con `STRIPE_WEBHOOK_SECRET` (`stripe.webhooks.constructEvent`) - sin firma válida, 400.
- Solo procesa `checkout.session.completed`.
- Es **idempotente**: si ya existe una orden con ese `stripe_payment_intent_id`, responde 200 sin duplicar.
- Usa `createServiceRoleClient()` (bypassa RLS) porque escribe en `orders`/`order_items`, tablas sin policy de `INSERT` pública.
- Inserta la orden con `subtotal_cents` recalculado de la metadata (nunca de lo que Stripe reporta como cobrado en cada línea), `shipping_cents`/`discount_cents`/`promo_code` leídos de la metadata, y `shipping_address_snapshot` desde `session.collected_information.shipping_details` (campo directo del objeto sesión en la API de Stripe usada - no requiere `expand`).
- Por cada ítem: inserta `order_items` con snapshot de nombre/atributos, decrementa `product_variants.stock_quantity`, e inserta un registro en `inventory_movements` con `reason: "sale"`.
- Si hubo `promo_code`, incrementa `promo_codes.times_used` (best-effort, no bloquea la creación del pedido si falla).

8.4. Confirmación (`(shop)/checkout/confirmacion/`)

- `getOrderBySessionId(sessionId)` - el `session_id` **de Stripe actúa como prueba de propiedad** para poder mostrarle la orden a un comprador invitado (sin cuenta) sin necesitar una policy de RLS adicional. Reintenta hasta 5 veces con 800 ms de espera, porque el webhook llega de forma asíncrona y puede no haber corrido todavía cuando el navegador redirige de vuelta.

9. Server Actions (catálogo completo)

Todas con `"use server"`; ninguna usa `service-role` salvo excepción explícita señalada.

Archivo	Acciones
`(shop)/checkout/actions.ts`	`createCheckoutSession`, `getCheckoutContext`, `previewPromoCode`
`(shop)/checkout/confirmacion/actions.ts`	`getOrderBySessionId` (usa service-role)
`(admin)/admin/actions.ts`	`updateOrderStatus`, `adjustStock`, `createProduct`, `updateProduct`, `deleteProduct`, `updateOrderTracking`, `markMessageRead`, `createPromoCode`, `updateUserRole`, `createBusiness` (valida que el dueño no sea admin y lo promueve a `vendor` antes de insertar), `updateBusiness`, `updateMyBusiness`
`(admin)/admin/pedidos/[id]/tracking-input.tsx` (client)	invoca `updateOrderTracking`
`(admin)/admin/usuarios/role-select.tsx` (client)	invoca `updateUserRole`
`(admin)/admin/mensajes/mark-read-button.tsx` (client)	invoca `markMessageRead`

9.1. Queries de solo lectura (`src/lib/supabase/queries.ts` y `admin-queries.ts`)

Listado no exhaustivo de las más relevantes: `getCategories`, `getCategoryShowcases`, `getFeaturedProducts`, `getProductsByCategory`, `getNewArrivals`, `getBestSellers`, `getProductBySlug`, `getRelatedProducts`, `getWishlistProducts`, `getVariantsForCheckout`, `getProductReviews`, `searchProducts`, `getUserOrders`,

`getAppSettings`, `validatePromoCode`, `getOrderDetail`, `getActiveBusinesses`, `getBusinessBySlug`, `getBusinessByOwnerId`, `getProductsByBusiness`, `getMyBusiness`; y del lado admin: `getAdminProducts`, `getAdminProduct`, `getLowStockVariants`, `getAllVariantsForInventory`, `getOrdersForRole`, `getContactMessages`, `getPromoCodes`, `getRevenueByDay`, `getTopProducts`, `getOrderStatusCounts`, `getAdminUsers`, `getBusinesses`, `getBusinessById`.

Los negocios pausados se excluyen del home, categorías, búsqueda y más vendidos mediante `getInactiveVendorIds` + `inactiveVendorFilter`.

9.2. Uso de `createServiceRoleClient()` (bypass de RLS) - únicamente en 3 lugares

1. `getBestSellers()` - agregado de ventas sitewide; `order_items` está RLS-restringido a comprador/vendor/admin, así que un visitante normal no puede calcular un ranking global. Falla de forma silenciosa (retorna `[]`) si `SUPABASE_SERVICE_ROLE_KEY` no está configurada, en vez de romper el home.
2. `validatePromoCode()` - `promo_codes` es admin-only por RLS; validar un código ingresado por el cliente necesita leer esa tabla. Mismo fallback silencioso (`{ valid: false }`) si falta la clave.
3. El webhook de Stripe (§8.3) y `getOrderBySessionId` - necesitan escribir/leer sin sesión de usuario autenticada.

`admin_list_users()` **es la excepción deliberada:** en vez de usar service-role para listar usuarios con email, se resolvió con una función Postgres `SECURITY DEFINER` - funciona incluso sin `SUPABASE_SERVICE_ROLE_KEY` configurada localmente.

10. Mapa completo de rutas

Tienda pública (`(shop)`)

`/`, `/categoria/[slug]`, `/producto/[slug]`, `/buscar`, `/carrito`, `/checkout`, `/checkout/confirmacion`, `/ayuda`, `/contacto`, `/terminos`, `/privacidad`, `/negocios`, `/negocio/[slug]`

Autenticación (fuera de grupos)

`/login`, `/signup`, `/recuperar`, `/restablecer`

Cuenta del cliente (`(account)`, requiere sesión)

`/cuenta`, `/cuenta/pedidos`, `/cuenta/pedidos/[id]`

Panel admin/vendedor (`(admin)`, requiere rol `admin` o `vendor`)

`/admin` (resumen), `/admin/productos`, `/admin/productos/nuevo`, `/admin/productos/[id]`, `/admin/inventario`, `/admin/pedidos`, `/admin/pedidos/[id]`, `/admin/analitica`, `/admin/mi-negocio`

Panel exclusivo admin (además del filtro anterior)

`/admin/mensajes`, `/admin/promociones`, `/admin/usuarios`, `/admin/negocios`, `/admin/negocios/[id]`

Rutas añadidas

- Tienda: `/vender`, `/comparar`, `/negocios`, `/negocio/[slug]`.
- Cuenta: `/cuenta/direcciones`, `/cuenta/devoluciones`, `/cuenta/devoluciones/[id]`, `/cuenta/privacidad`.
- Panel (admin y vendedor): `/admin/devoluciones`, `/admin/devoluciones/[id]`, `/admin/preguntas`, `/admin/mi-negocio` (vendedor).
- Panel (solo admin): `/admin/solicitudes`, `/admin/negocios`, `/admin/resenas`, `/admin/correos`, `/admin/bitacora`, `/admin/privacidad`.
- API: `GET /api/account/export` (descarga de datos del usuario), `GET|POST /api/cron/send-emails` (protegida con `CRON_SECRET`).

API

`POST /api/webhooks/stripe`

Todas las rutas anteriores existen bajo `/es/...` y `/en/...` (prefijo de locale obligatorio).

11. Gestión de estado en cliente (Zustand)

- `cart-store.ts` - carrito persistido en `localStorage` (`omnara-cart`), con `skipHydration: true` + hook `useCartHydrated()` para evitar parpadeo de hidratación SSR/CSR. El carrito **no** usa las tablas `carts`/`cart_items` de Supabase en el flujo actual - esas tablas existen en el esquema pero el carrito real vive en el navegador hasta el checkout.
- `currency-store.ts` - moneda activa (MXN/USD) y `usdExchangeRate`, usada por `useDisplayPrice` para recalcular cada precio mostrado en la UI.

12. Variables de entorno

Variable	Requerida para	¿Configurada en `.env.local` local?
`NEXT_PUBLIC_SUPABASE_URL`	Cliente Supabase (browser + server)	Sí
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Cliente Supabase	Sí
`SUPABASE_SERVICE_ROLE_KEY`	`getBestSellers`, `validatePromoCode`, webhook, confirmación de checkout	**No** - degradan de forma controlada (ver §9.2)
`STRIPE_SECRET_KEY`	Cualquier operación de Stripe (`getStripe()` lanza error si falta)	**No**
`STRIPE_WEBHOOK_SECRET`	Verificación de firma del webhook	**No**
`RESEND_API_KEY`, `EMAIL_FROM`	Envío de correos (sin ellos quedan en cola)	No
`CRON_SECRET`	Protege `/api/cron/send-emails`	No
`NEXT_PUBLIC_SITE_URL`	Enlaces dentro de los correos	No
`NEXT_PUBLIC_TURNSTILE_SITE_KEY`, `TURNSTILE_SECRET_KEY`	CAPTCHA (Cloudflare Turnstile); activar también en el panel de Supabase	No

`.env.example` documenta las 5. Ni `.env.local` ni `.env.example` están versionados en git (`.gitignore`: `.env*`).

13. Seguridad y buenas prácticas verificadas

- Auditoría completa del historial de git (no solo el HEAD) sin hallazgos de secretos reales filtrados - únicamente `.env.example` (vacío) y `.claude/launch.json` (solo el comando `npm run dev`) estuvieron versionados alguna vez, y se destrackearon.
- `.gitignore` cubre `.env*`, `.vercel`, `.claude`, y un set amplio de certificados/llaves (`*.pem`, `*.key`, `*.crt`, `*.cer`, `*.p12`, `*.pfx`, `*.der`, `.env.vault`).
- El repositorio de GitHub es **público** (`jp-software-dev/omnara-ecommerce`).

14. Despliegue

- **Vercel**, proyecto `omnara-ecommerce` (`prj_3wLsc1N7opx6Nly7bIxp4kB5KicP`), integración Git nativa: cada push a `main` dispara un deploy de producción; cada rama/PR genera un preview deployment automático.
- Dominios: `omnara-ecommerce.vercel.app` (+ alias por equipo/rama).
- Build con Turbopack (`bundler: turbopack` en la metadata de cada deployment).
- **Supabase** aloja Postgres/Auth/Storage. Todo el esquema está versionado como SQL en `supabase/migrations` (tablas y políticas originales más los cambios posteriores); se aplica en orden de nombre de archivo sobre un proyecto vacío. **El proyecto de Supabase es compartido**: además de Omnara (esquema `public`) aloja los esquemas `textiles` y `mediagenda` de otras aplicaciones, cuyas migraciones no se incluyen; tampoco se incluyen los datos de demostración del catálogo (ver `supabase/migrations/README.md`).

15. Patrones de código recurrentes (para mantener consistencia futura)

- **Money:** enteros en centavos, `formatPrice(cents, currency, locale)` (`src/lib/format.ts`) para mostrar.
- **Textos bilingües en BD:** JSONB `{es, en}` + `pickLocale()`.

- **Textos de UI fuera de BD:** objeto `t` local por componente/página con ternario `locale === "en" ? ... : ...`.
- **Controles optimistas admin** (estado de pedido, rol de usuario, tracking number): `useState` local + `useTransition`, revierte en `catch` y muestra `toast.error`.
- **Guards de acceso:** `requireRole()` en Server Components de página; nunca se confía solo en ocultar el link de navegación.
- **Nunca se confía en el cliente para precio/descuento/stock** - todo se revalida server-side antes de tocar Stripe o la BD.

16. Calidad: pruebas, integración continua y migraciones

- **Pruebas unitarias** (Vitest, `npm test`): 32 casos sobre envío, conversión y formato de moneda, validación de productos, negocios, códigos promocionales y direcciones, la ventana de 30 días de devoluciones y el renderizado de correos (incluido el escape de HTML).
- **CI** (`.github/workflows/ci.yml`): `npm run lint`, `npm run typecheck` y `npm test` en cada pull request y en cada push a `main` y `development`. El build de producción lo hace Vercel.
- **Reglas de base de datos:** se verificaron con scripts SQL dentro de transacciones que se revierten (límites, permisos, disparadores, flujos completos); no forman parte de la CI.
- **Lockfile:** se fijaron `@emnapi/core` y `@emnapi/runtime` para que `npm ci` funcione en Linux con un lockfile generado en Windows.

17. Seguridad y privacidad añadidas

- **Validación en servidor** con zod (`src/lib/validation.ts`, `src/lib/addresses.ts`) en productos, negocios, códigos promocionales, variantes, direcciones, devoluciones, preguntas y contacto.
- **Contacto:** sin INSERT público directo; `submit_contact_message` valida y limita a 3 mensajes por correo y 40 en total por hora; la acción del servidor añade un campo trampa y, si hay claves, verifica el CAPTCHA.
- **CAPTCHA** (Cloudflare Turnstile): el componente no hace nada sin `NEXT_PUBLIC_TURNSTILE_SITE_KEY`; la política CSP solo permite el dominio de Cloudflare cuando hay clave.
- **Bitácora de administradores:** disparadores que registran quién cambió qué columnas, solo para acciones de administradores, sin copiar datos personales y sin posibilidad de editar o borrar.
- **Almacenamiento:** el bucket `product-images` limita tamaño (5 MB) y tipos, y cada vendedor solo escribe en su carpeta; `seller-documents` es privado y solo lo leen administradores.
- **Privacidad:** aviso y términos reales, exportación de datos del usuario y solicitudes de eliminación atendidas por un administrador.
- **Ajustes que solo se hacen en el panel de Supabase:** protección contra contraseñas filtradas y CAPTCHA de autenticación.